

FinPilot — Complete Project Context

Who the user is

- B.Tech student (graduating 2027), intermediate Python + ML/AI knowledge
 - **Complete beginner to Java and backend development**
 - Learning approach: user studies concepts first → builds themselves → asks for help only when stuck
 - **The assistant should guide and explain, never write the code for them**
 - Dev setup: Windows, VS Code installed. Still needs: Java 21 (Temurin), Maven, Docker Desktop, Node.js 20
-

What FinPilot is

An AI-powered personal finance web app built across 5 phases. The user is building this project specifically to learn Java + Spring Boot backend development, while also integrating their existing ML/AI skills in later phases.

Think of it as a smart financial co-pilot — dashboards, alerts, reports, and autonomous background agents.

Full Tech Stack

Layer	Technology
Frontend	React 18 + Vite + TailwindCSS (mobile-first PWA)
Backend	Java 21, Spring Boot 3, Spring Security 6
Database	PostgreSQL 16 + pgvector (later phases)
Auth	JWT (jjwt library) — stateless, no sessions
ML	Python microservice via REST (Phase 3)
RAG / Agents	Spring AI (Phase 4 + 5)
Cache	Redis (Phase 4+)
LLM	Claude API / OpenAI API (Phase 4+)

Layer	Technology
DevOps	Docker Compose (local), Maven

The 5 Phases

Phase 1 — Foundation (Weeks 1-2) ← CURRENT PHASE

Goal: A working full-stack app. No AI yet.

What gets built:

- Spring Boot 3 backend running on `localhost:8080`
- PostgreSQL via Docker Compose
- JWT auth: `POST /auth/register`, `POST /auth/login`
- Transaction CRUD API: `GET/POST/PUT/DELETE /api/transactions`
- Monthly summary endpoint: `GET /api/transactions/summary` → returns `totalIncome`, `totalExpense`, `savingsRate`
- React frontend on `localhost:5173`
- Login + Signup screens (call Spring Boot auth endpoints, store JWT)
- Protected routes — dashboard only accessible when logged in
- Dashboard screen: net worth, monthly burn rate, savings rate, transaction list, add/edit/delete form, monthly spend bar chart (Recharts)

User journey: Open app → Sign up / Log in → Add transactions manually → See live dashboard

Project folder structure:

```

finpilot/
├── backend/                                ← Spring Boot
│   ├── src/main/java/com/finpilot/
│   │   ├── auth/                          ← register, login, JWT
│   │   ├── user/                          ← User entity + repo
│   │   ├── transaction/                   ← CRUD API
│   │   └── config/                        ← Security config
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── docker-compose.yml
│   └── frontend/                          ← React + Vite
│       ├── src/
│       │   ├── pages/                     ← Login, Dashboard
│       │   ├── components/                ← cards, charts
│       │   └── api/                       ← Axios calls
│       └── index.html

```

Phase 1 — 18-task checklist:

Setup:

- ☐ Java 21, Maven, Docker Desktop, Node installed — verify with version commands
- ☐ VS Code: Extension Pack for Java + Spring Boot Extension Pack installed

Backend — Auth:

- ☐ Spring Boot project created, runs on localhost:8080
- ☐ PostgreSQL running via Docker Compose (`docker ps` shows container)
- ☐ Spring Boot connects to PostgreSQL on startup (no errors in logs)
- ☐ User entity created — `users` table auto-created in DB by Hibernate
- ☐ POST `/auth/register` works — creates a user row in DB
- ☐ POST `/auth/login` works — returns a JWT token
- ☐ Protected route returns 401 without token, 200 with valid token

Backend — Transactions:

- ☐ Transaction entity created — `transactions` table with amount, date, category, note, user
- ☐ POST `/api/transactions` — add a transaction (auth required)
- ☐ GET `/api/transactions` — returns only the logged-in user's transactions
- ☐ PUT `/api/transactions/{id}` and DELETE `/api/transactions/{id}` work
- ☐ GET `/api/transactions/summary` — returns totalIncome, totalExpense, savingsRate

Frontend:

- ☐ React + Vite + Tailwind running on localhost:5173
- ☐ Login screen — calls /auth/login, stores JWT, redirects to dashboard
- ☐ Signup screen — calls /auth/register, then auto-logs in
- ☐ Dashboard shows real data from API (net worth, burn rate, savings rate)
- ☐ Add transaction form submits to API, list refreshes without page reload

What is NOT in Phase 1: PDF upload, AI/ML, forecasts, RAG, agents, risk score, push notifications

Phase 2 — Statement Ingestion (Weeks 3-4)

- File upload endpoint
- Apache PDFBox for PDF bank statement parsing
- Rule-based classifier + Naive Bayes model for auto-categorisation (Food, Rent, EMI, etc.)
- Statement upload UI
- **No Spring AI yet**
- Ships: Upload a bank PDF → transactions auto-appear categorised

Phase 3 — ML Predictions (Weeks 5-6)

- LSTM model for 30-day spend forecasting
- Random Forest for risk scoring
- Python ML microservice via REST (called from Spring Boot)
- Spring @Scheduled jobs
- Forecast chart screen
- Spring AI possibly used for simple LLM insights
- Ships: Dashboard shows "you'll overspend ₹4K this month" with bar chart. Risk badge updates weekly.

Phase 4 — RAG Document Q&A (Weeks 7-8) ← Spring AI enters here

- Text chunking strategy
- OpenAI embeddings API
- pgvector setup in PostgreSQL
- **Spring AI** — ChatClient, EmbeddingModel, VectorStore, RAG pipeline
- Claude API integration
- Chat-style Q&A screen
- Ships: Ask "what did I spend on subscriptions in March?" → cited answer from your own PDF

Phase 5 — Agentic Engine (Weeks 9-10) ← Spring AI used heavily

- @Scheduled background jobs
 - Multi-step agent loop (plan → retrieve → act)
 - **Spring AI** @Tool annotation, function calling
 - Push notification / email report
 - Agent log screen
 - Ships: Every Sunday, agent runs automatically — parses data, forecasts, compares benchmarks, sends PDF report
-

Spring AI placement

- **P1, P2:** No Spring AI
 - **P3:** Maybe (optional LLM insights)
 - **P4:** Spring AI enters — ChatClient, EmbeddingModel, VectorStore, RAG pipeline
 - **P5:** Spring AI heavily used — @Tool, function calling, agent loop
-

Phase 1 Learning Roadmap

Week 1 — Backend

1. **Java fundamentals** (learn) — classes, types, interfaces, annotations. Just enough to read Spring code.
2. **Spring Boot project setup** (build) — create on start.spring.io, understand folder structure, run Hello World
3. **PostgreSQL via Docker** (build) — write docker-compose.yml, connect to Spring Boot via application.yml
4. **User entity + JPA** (learn then build) — @Entity, @Id, @Column, JpaRepository
5. **JWT auth concept** (learn) — understand the full flow before touching code
6. **Build auth endpoints** (build) — register, login, JWT filter chain, Spring Security config
7. **Transaction CRUD API** (build) — entity, repository, service, controller, summary endpoint

Week 2 — Frontend

1. **React + JS basics** (learn) — components, useState, useEffect, props
2. **Vite + Tailwind scaffold** (build) — create React project, get it running

3. **Login + signup screens** (build) — forms, call auth endpoints, store JWT
4. **React Router + protected routes** (learn then build) — navigation, block unauthenticated users
5. **Dashboard screen** (build) — fetch real data from Spring Boot API, Recharts for bar chart

Key learning resources

- Java: <https://www.baeldung.com/java-tutorial> — Baeldung Java basics
 - Spring Boot: <https://spring.io/guides/gs/rest-service>
 - JPA: <https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa>
 - React: <https://react.dev/learn>
-

Language decision — Java not Go

User asked about replacing Java with Go. Decision: **stick with Java** because:

- Spring AI (Phase 4+5) has no Go equivalent — would require raw HTTP calls, manual embeddings, manual vector search
 - Go would mean learning Go instead of Java, defeating the project's purpose
 - Spring Security handles JWT auth in one config class — Go requires assembling middleware manually
 - Go is better suited for high-performance microservices or CLI tools, not AI-heavy full-stack apps
-

Assistant behaviour rules for this project

- **Never write the code for the user** — guide, explain, point in the right direction
- When explaining Java/Spring concepts, compare to Python equivalents where helpful
- When user is stuck: help debug or explain the concept, don't hand them the solution
- Explain every concept before the user touches code for it (especially JWT, JPA, Spring Security)
- Break things into small, testable pieces — one working thing at a time